

# Automaten und Sprachen | AutoSpr

## Zusammenfassung

### INHALTSVERZEICHNIS

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>1. Prädikate</b>                                 | <b>3</b>  |
| 1.1. Normalformen                                   | 3         |
| 1.2. Negation                                       | 3         |
| <b>2. Mengen</b>                                    | <b>3</b>  |
| 2.1. Konstruktion                                   | 3         |
| 2.2. Operationen                                    | 3         |
| 2.3. Tupel                                          | 3         |
| <b>3. Beweise</b>                                   | <b>4</b>  |
| <b>4. Sprachen</b>                                  | <b>4</b>  |
| 4.1. Wortlänge                                      | 4         |
| 4.2. Deterministische Endliche Automaten (DEA)      | 5         |
| 4.3. Wörter einer regulären Sprache                 | 5         |
| 4.3.1. Übergangsfunktionen für Wörter               | 5         |
| 4.3.2. Wort akzeptieren                             | 5         |
| 4.3.3. Akzeptierte Sprache, reguläre Sprache        | 5         |
| 4.4. Myhill-Nerode Automat                          | 6         |
| 4.5. Minimalautomat                                 | 6         |
| 4.6. Pumping Lemma                                  | 7         |
| 4.6.1. Beweis                                       | 7         |
| 4.7. Nichtdeterministische endliche Automaten (NEA) | 8         |
| 4.7.1. Umgang mit mehrdeutigen Übergängen           | 9         |
| 4.7.2. Thompson-NEA                                 | 9         |
| 4.7.3. $NEA_{\epsilon}$                             | 9         |
| 4.8. Mengenoperationen                              | 10        |
| 4.9. Reguläre Operationen                           | 11        |
| 4.9.1. Alternative                                  | 11        |
| 4.9.2. Verkettung                                   | 12        |
| 4.9.3. *-Operation                                  | 12        |
| 4.10. Reguläre Ausdrücke                            | 12        |
| 4.10.1. Verallgemeinerter NEA (VNEA)                | 12        |
| 4.10.2. Teststrategie                               | 14        |
| 4.11. Kontextfreie Sprachen                         | 14        |
| 4.11.1. Kontextfreie Grammatik und Sprache          | 14        |
| 4.11.2. Mehrere Grammatiken                         | 15        |
| 4.11.3. Reguläre Operationen                        | 15        |
| 4.11.4. Kontextfrei                                 | 15        |
| 4.11.5. Ableitung                                   | 15        |
| 4.11.6. Chomsky-Normalform (CNF)                    | 16        |
| 4.11.7. Cocke-Younger-Kasami Algorithmus (CYK)      | 17        |
| <b>5. Stack</b>                                     | <b>17</b> |
| 5.1. Stackautomat / Pushdown Automaton (PDA)        | 18        |

|                                             |           |
|---------------------------------------------|-----------|
| 5.1.1. Ableitung aus CNF .....              | 18        |
| 5.1.2. PDA $\rightarrow$ CFG .....          | 19        |
| <b>6. Parsing .....</b>                     | <b>21</b> |
| 6.1. Backus-Naur-Form (BNF) .....           | 21        |
| 6.2. Extended Backus-Naur-Form (EBNF) ..... | 22        |
| 6.3. Rechtsrekursion .....                  | 22        |
| <b>7. Unendlich .....</b>                   | <b>23</b> |

# 1. PRÄDIKATE

Prädikate sind Aussagen über mathematische Objekte, die wahr oder falsch sein können. «Funktionen» mit booleschen Rückgabewerten:  $P, Q(n), R(x, y, z)$ . Siehe [DMI](#).

## 1.1. NORMALFORMEN

Normalformen (allgemein, *kanonische Formen*) helfen, das Vergleichsproblem zu lösen (ob zwei Aussagen dieselben sind).

$$P_1 \wedge \dots \wedge P_n = \forall i \in \{1, \dots, n\} (P_i) = \bigwedge_{i=1}^n P_i$$

$$P_1 \vee \dots \vee P_n = \exists i \in \{1, \dots, n\} (P_i) = \bigvee_{i=1}^n P_i$$

## 1.2. NEGATION

Nicht für alle = Es gibt einen Fall, für den nicht

$$\neg \forall i \in \{1, \dots, n\} (P_i) \Leftrightarrow \neg (P_1 \wedge \dots \wedge P_n) \Leftrightarrow \neg P_1 \vee \dots \vee \neg P_n \Leftrightarrow \exists i \in \{1, \dots, n\} (\neg P_i)$$

# 2. MENGEN

## 2.1. KONSTRUKTION

Grundmenge  $G$ , Prädikat  $P(x)$ . Konstruierte Menge  $A = \{x \in G \mid P(x)\}$

## 2.2. OPERATIONEN

| Begriff      | Bedeutung                                      |
|--------------|------------------------------------------------|
| Vereinigung  | $A \cup B = \{x \mid x \in A \vee x \in B\}$   |
| Schnittmenge | $A \cap B = \{x \mid x \in A \wedge x \in B\}$ |
| Komplement   | $\overline{A} = \{x \mid x \notin A\}$         |
| Differenz    | $A \setminus B = \{x \in A \mid x \notin B\}$  |

## 2.3. TUPEL

$$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$$

$n$ -Tupel:

$$\bigtimes_{i=1}^n A_i = \{(a_1, a_2, \dots, a_n) \mid a_i \in A_i\}$$

### 3. BEWEISE

Eine Folge von logischen Schlüssen, die zeigen, dass die Aussage aus den gegebenen Voraussetzungen folgt.

| <i>Beweistechnik</i>   | <i>Anwendungsfall</i>                                                  |
|------------------------|------------------------------------------------------------------------|
| Konstruktiver Beweis   | Liefert explizite «Lösung» / Algorithmus                               |
| Widerspruchsbeweis     | Geeignet für Unmöglichkeitsaussagen. Z.B. $\sqrt{2} \notin \mathbb{Q}$ |
| Vollständige Induktion | Für Aussagen, die für alle $n \in \mathbb{N}$ gelten.                  |

### 4. SPRACHEN

| <i>Begriff</i>                       | <i>Bedeutung</i>                                                                                                                                                                                        |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Alphabet                             | Eine nichtleere endliche Menge $\Sigma$ heisst <b>Alphabet</b> . Die Elemente von $\Sigma$ heissen <b>Zeichen</b> .                                                                                     |
| Wort                                 | Eine Zeichenkette der Länge $n$ ist ein $n$ -Tupel in $\Sigma^n = \Sigma \times \dots \times \Sigma$ . Ein Element von $\Sigma^n$ heisst <b>Wort</b> der Länge $n$ . Wörter haben immer endliche Länge. |
| Leeres Wort                          | Die Zeichenkette $\varepsilon \in \Sigma^0 = \{\varepsilon\}$ der Länge 0 heisst das <b>leere Wort</b> .                                                                                                |
| Menge aller Wörter                   | Leeres Wort ist <b>immer</b> drin<br>$\Sigma^* = \{\varepsilon\} \cup \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots = \bigcup_{k=0}^{\infty} \Sigma^k$                                                  |
| Abgekürzte Schreibweisen von Wörtern | $(A, u, t, o, S, p, r) = AutoSpr$<br>$a^3 b^5 = aaabbbbbb$<br>$b^5 a^3 = bbbbaaaa$                                                                                                                      |
| Sprache                              | Teilmenge $L \subset \Sigma^*$                                                                                                                                                                          |

#### 4.1. WORTLÄNGE

| <i>Begriff</i>   | <i>Bedeutung</i>                                                                                   |
|------------------|----------------------------------------------------------------------------------------------------|
| Länge des Wortes | $w \in \Sigma^n \Rightarrow  w  = n$ , $n$ ist <b>Länge des Wortes</b> $w$                         |
| Anzahl Zeichen   | Sei $w \in \Sigma^n$ , $a \in \Sigma$ . Dann ist $ w _a$ die <b>Anzahl Zeichen</b> $a$ im Wort $w$ |

#### Beispiel 1 : Wortlänge

$$|\varepsilon| = 0$$

$$|01010|_0 = 3$$

$$|01010|_1 = 2$$

$$|01010|_3 = 0$$

$$|a^3 b^5| = 8$$

$$|(1291)^7| = 7|1291| = 7 \cdot 4 = 28$$

**Beispiel 2 : Sprache**

$$L = \Sigma^* \subset \Sigma^*$$

$L = \emptyset \subset \Sigma^*$ , die leere Sprache

$\Sigma = \{0, 1\}$ , Sprache aller Binärstrings:  $L = \Sigma^*$

$\Sigma = \text{Unicode}$ ,  $J = \{w \in \Sigma^* \mid w \text{ wird vom Java-Compiler akzeptiert}\}$

$M$  eine "Maschine",  $L(M) = \{w \in \Sigma^* \mid w \text{ wird von } M \text{ akzeptiert}\}$

**Beobachtungen 1**

1.1.  $|w^n| = n \cdot |w|$

1.2. Zu jeder Maschine  $M$  gibt es eine Sprache  $L(M)$

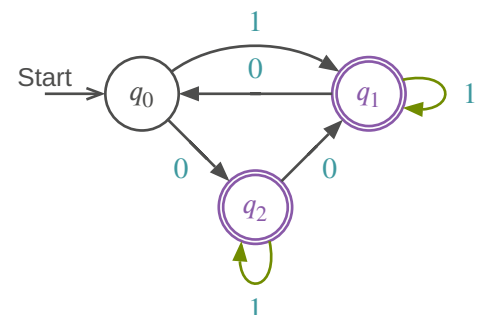
**4.2. DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)****Definition**

$$A = (\mathcal{Q}, \Sigma, \delta, q_0, F)$$

- Zustände:  $\mathcal{Q} = \{q_0, q_1, \dots, q_n\}$
- Alphabet:  $\Sigma$
- Übergangsfunktion:  $\delta : \mathcal{Q} \times \Sigma \rightarrow \mathcal{Q}$
- Startzustand:  $q_0 \in \mathcal{Q}$
- Akzeptierzustände:  $F \subset \mathcal{Q}$

**Tabellenform**

|       | 0     | 1     | $\Sigma$ |
|-------|-------|-------|----------|
| $q_0$ | $q_2$ | $q_1$ | $\delta$ |
| $q_1$ | $q_0$ | $q_1$ |          |
| $q_2$ | $q_1$ | $q_2$ |          |

**Zustandsdiagramm von A****Wichtig:**

- Ein Pfeil für jedes Zeichen in jedem Zustand (deterministischer endlicher Automat / DEA)

**4.3. WÖRTER EINER REGULÄREN SPRACHE****4.3.1. Übergangsfunktionen für Wörter**

$$\delta : \mathcal{Q} \times \Sigma^* \rightarrow \mathcal{Q} : (q, w = a_1, \dots, a_n) \mapsto \delta(\dots \delta(\delta(q, a_1), a_2), \dots, a_n)$$

**Übergänge** ausgehend von  $q$  für Zeichen  $a_1, \dots, a_n$  nacheinander anwenden.

**4.3.2. Wort akzeptieren**

Der DEA  $A = (\Sigma, \mathcal{Q}, q_0, \delta, F)$  **akzeptiert** das Wort  $w \in \Sigma^*$ , wenn er  $A$  von Startzustand in einen Akzeptierzustand  $\delta(q_0, w) \in F$  überführt.

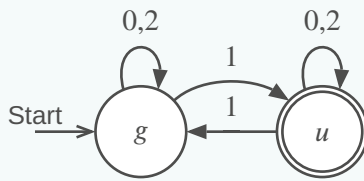
**4.3.3. Akzeptierte Sprache, reguläre Sprache**

Gegeben ein DEA  $A = (\Sigma, \mathcal{Q}, q_0, \delta, F)$ . Die von  $A$  **akzeptierte Sprache** ist

$$L(A) = \{w \in \Sigma^* \mid A \text{ akzeptiert } w\} = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$$

Die Sprache  $L \subset \Sigma^*$  heisst **regulär**, wenn es einen DEA  $A$  gibt mit  $L(A) = L$

**Beispiel 3:** DEA, der die Sprache  $L = \{w \in \Sigma^* \mid w \text{ ist eine ungerade Zahl im Dreiersystem}\}$  akzeptiert



|   | 0 | 1 | 2 |
|---|---|---|---|
| g | g | u | g |
| u | u | g | u |

#### 4.4. MYHILL-NERODE AUTOMAT

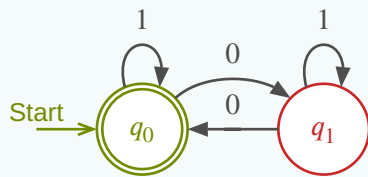
Für ein Wort  $w \in \Sigma^*$  setze  $L(w) = \{w' \in \Sigma^* \mid ww' \in L\}$ . Insbesondere  $L(\varepsilon) = L$

Gegeben: reguläre Sprache  $L$  über  $\Sigma$ . Rekonstruiere  $A$  mit:

- $Q = \{L(w) \mid w \in \Sigma^*\}$
- $q_0 = L(\varepsilon) = L$
- $F = \{L(w) \in Q \mid \varepsilon \in L(w)\}$
- $\delta(L(w), a) = L(wa)$

Gleicher Zustand  $\Leftrightarrow$  gleiches  $L(w)$

**Beispiel 4:**  $\Sigma = \{0, 1\}$ ,  $L = \{w \in \Sigma^* \mid |w|_0 \text{ gerade}\}$



| $w$           | $L(w)$                                                  | $Q$      |
|---------------|---------------------------------------------------------|----------|
| $\varepsilon$ | $L(\varepsilon) = L$                                    | $q_0$    |
| 0             | $L(0) = \{w \in \Sigma^* \mid  w _0 \text{ ungerade}\}$ | $q_1$    |
| 1             | $L(1) = \{w \in \Sigma^* \mid  w _0 \text{ gerade}\}$   | $q_0$    |
| $\vdots$      | $\vdots$                                                | $\vdots$ |

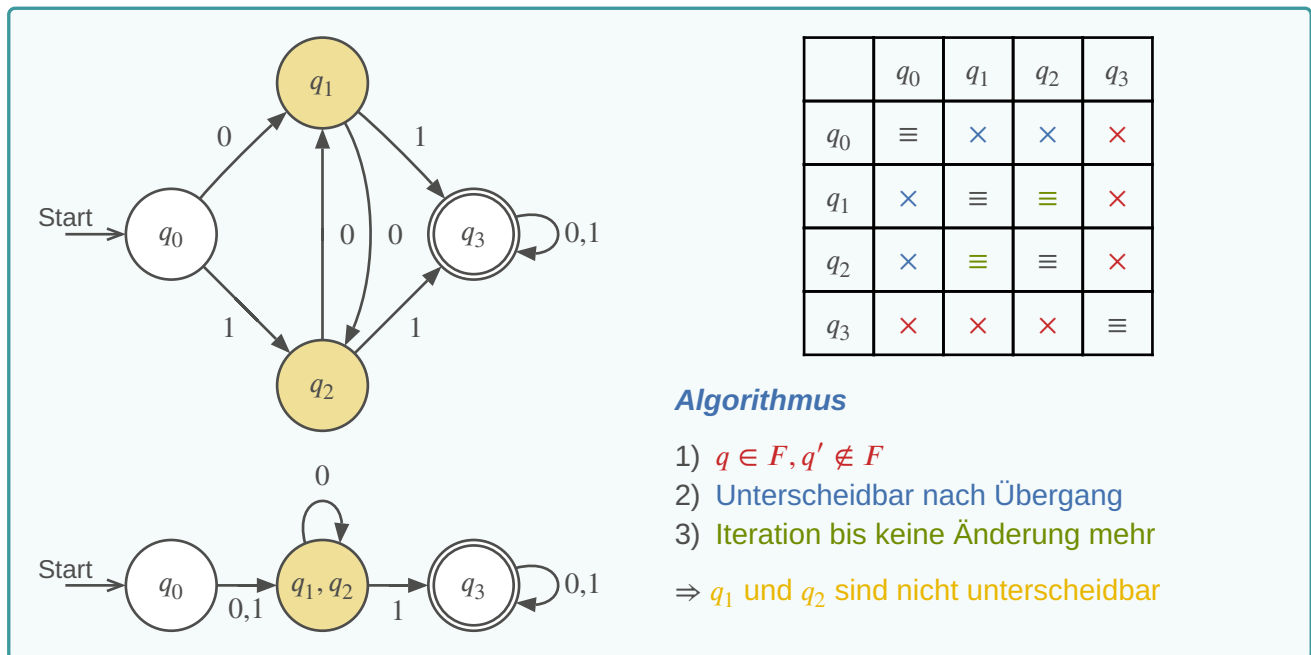
#### 4.5. MINIMALAUTOMAT

**Ziel:** Nicht unterscheidbare Zustände zusammenlegen

**Vorgehen:**

- 1) Akzeptierzustand  $\neq$  Nichtakzeptierzustand
- 2) Iteration: alle unterscheidbaren Paare suchen
- 3) Verbleibende Paare sind ununterscheidbar  $\rightarrow$  zusammenlegen

**Beispiel 5**

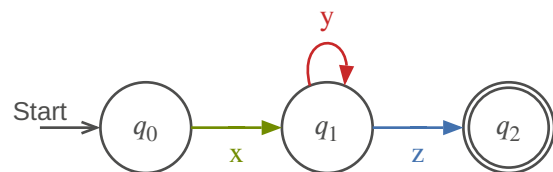


## 4.6. PUMPING LEMMA

Ist  $L$  eine reguläre Sprache, dann gibt es  $N \in \mathbb{N}$ , die pumping length so, dass jedes Wort  $w \in L$  mit  $|w| \geq N$  in drei Teile  $w = xyz$  zerlegt werden kann mit:

- 1)  $|xy| \leq N$
- 2)  $|y| > 0$
- 3)  $xy^kz \in L \forall k \in \mathbb{N}$

Oder: genügend lange Wörter ( $|w| \geq N$ ) einer regulären Sprache ( $w \in L$ ) können alle in einem Anfangsstück der Länge  $N$  ( $|xy| \leq N$ ) aufgepumpt werden ( $xy^kz \in L$ ).



Was zeichnet reguläre Sprachen aus?

- Reguläre Ausdrücke
- Lange Wörter:  $*$  kommt im regulären Ausdruck vor
- Blöcke mit  $*$  können beliebig oft wiederholt werden
- $\Rightarrow$  Wörter können «aufgepumpt» werden

### 4.6.1. Beweis

Behauptung: Die Sprache  $L \subset \Sigma^*$  ist nicht regulär.

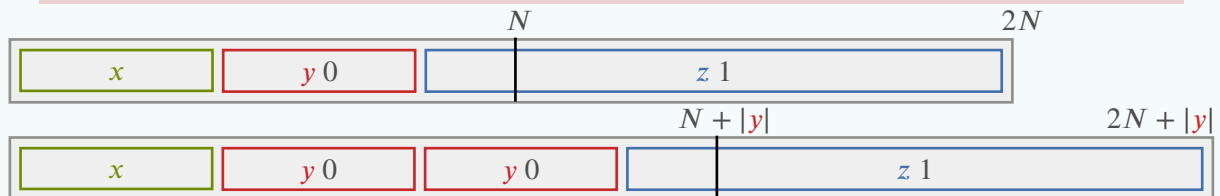
Beweis (Widerspruch):

- 1) Annahme:  $L$  ist regulär
- 2) Gemäss Pumping Lemma gibt es die Pumping Length  $N$ 
  - Es darf keine Annahme über die konkrete Grösse von  $N$  gemacht werden!
- 3) Wähle ein Wort  $w \in L$  mit  $|w| \geq N$ 
  - Die Definition **muss**  $N$  verwenden!
- 4) Aufteilung des Wortes gemäss Pumping Lemma
  - $w = xyz, |xy| \leq N, |y| > 0$
- 5) Auswirkung des Pumpens
  - $xy^kz \notin L$  für mindestens ein  $k \in \mathbb{N}$  (mit Begründung!)
- 6) Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

**Beispiel 6:**  $L = \{0^n 1^n \mid n \geq 0\}$  ist nicht regulär

- 1) Annahme:  $L$  ist regulär
- 2)  $\exists N \in \mathbb{N}$ , Pumping Length
- 3)  $w = 0^N 1^N$
- 4) Unterteilung  $w = xyz$

**TODO:**  
reduce confusion



- 5) Pumpen: nur die Anzahl der 1 wird erhöht, Anzahl 1 bleibt
- 6)  $xy^kz \notin L$  für  $k \neq 1$ , im Widerspruch zum Pumping Lemma

**4.7. NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)****Definition**

$$A = (Q, \Sigma, \delta, q_0, F)$$

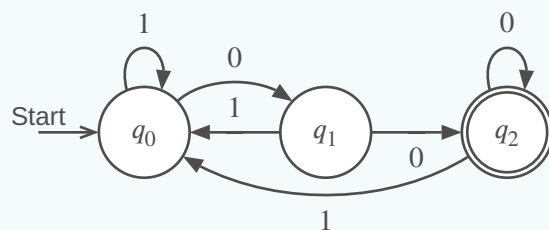
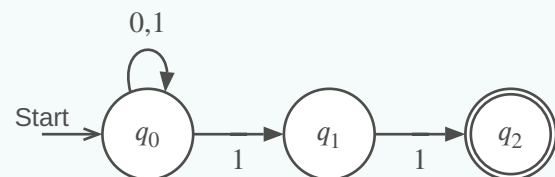
- Endliche Menge von Zuständen:  $Q$
- Alphabet:  $\Sigma$
- Übergangsfunktion:  $\delta : Q \times \Sigma \rightarrow P(Q)$
- Startzustand:  $q_0 \in Q$
- Akzeptierzustände:  $F \subset Q$

**Akzeptieren**

Ein NEA  $A$  **akzeptiert** das Wort  $w \in \Sigma^*$ , wenn es eine **Wahl** von Übergängen gibt, derart, dass das Wort  $w$  den Automaten in einen Akzeptierzustand überführt.

**Faustregeln**

- Nur genau diejenigen Pfeile einzeichnen, die man zum Akzeptieren braucht.
- Erlaube Alternativen

**Beispiel 7:**  $L = \{w \in \Sigma^* \mid w \text{ endet mit zwei } 0\}$ **DEA-Lösung****NEA-Lösung**

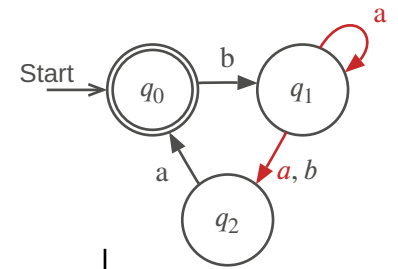


### 4.7.1. Umgang mit mehrdeutigen Übergängen

#### a-Übergang von $q_1$ aus nicht eindeutig

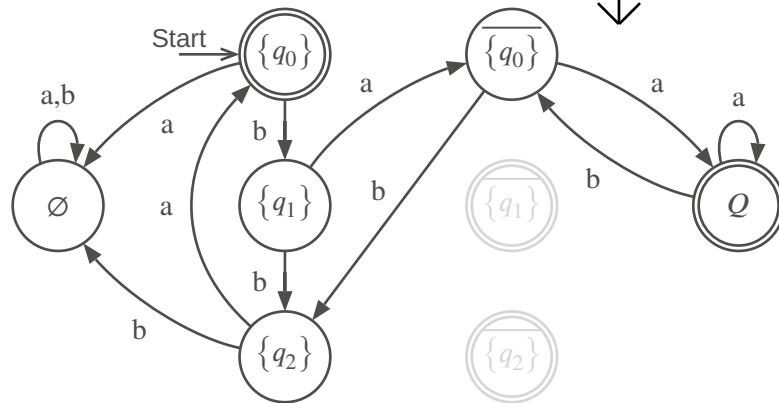
Welchen Übergang soll man nehmen?

- Beide Möglichkeiten probieren und akzeptieren, falls eine der Möglichkeiten auf einen Akzeptierzustand führt.
- Zufallsgenerator (probabilistischer endlicher Automat, PEA)



### 4.7.2. Thompson-NEA

- Zustand = **Menge** der möglichen Zustände
- Akzeptieren = Ob NEA im Zustand sein **könnte**



#### 4.7.2.1. Transformation NEA → DEA

Gegeben  $\delta : Q \times \Sigma \rightarrow P(Q)$  eines NEA. Übergangsfunktion für Mengen  $M \subset Q$

$$\delta' : P(Q) \times \Sigma \rightarrow P(Q) : (M, a) \mapsto \delta'(M, a) = \bigcup_{q \in M} \delta(q, a)$$

#### Satz

Ein NEA  $A$  kann in einen DEA  $A'$  umgewandelt werden mit

- $Q' = P(Q)$
- $\Sigma' = \Sigma$
- $\delta'$  wie oben definiert
- $q'_0 = \{q_0\}$
- $F' = \{M \in P(Q) \mid F \cap M \neq \emptyset\}$

#### Implementation

Thompson-NEA:

- Zustand  $M \subset Q$  realisiert durch Markierung der Zustände in  $M$
- $\delta'$  realisiert durch Verschieben von Markierungen
- Akzeptieren: mindestens ein Akzeptierzustand markiert

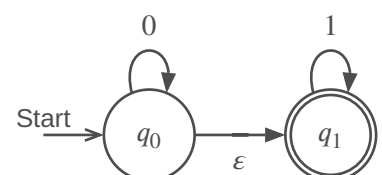
### 4.7.3. $NEA_\epsilon$

| Begriff              | Bedeutung                                                                                                                 |
|----------------------|---------------------------------------------------------------------------------------------------------------------------|
| $\epsilon$ -Übergang | Kann ohne Verarbeitung eines Zeichens genommen werden.<br>$\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow P(Q)$ |
| $NEA_\epsilon$       | NEA mit $\epsilon$ -Übergängen                                                                                            |

Einen  $NEA_\epsilon$  kann man immer in einen NEA umwandeln. Beweis:

- 1)  $E(q)$  = Menge der von  $q$  aus mit  $\epsilon$ -Übergängen erreichbaren Zustände
- 2)  $E(M) = \bigcup_{q \in M} E(q)$
- 3)  $\delta$  ersetzen durch  $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow P(Q) : (q, a) \mapsto E(\delta(q, a))$

$$L = \{0^k 1^l \mid k, l \geq 0\}$$

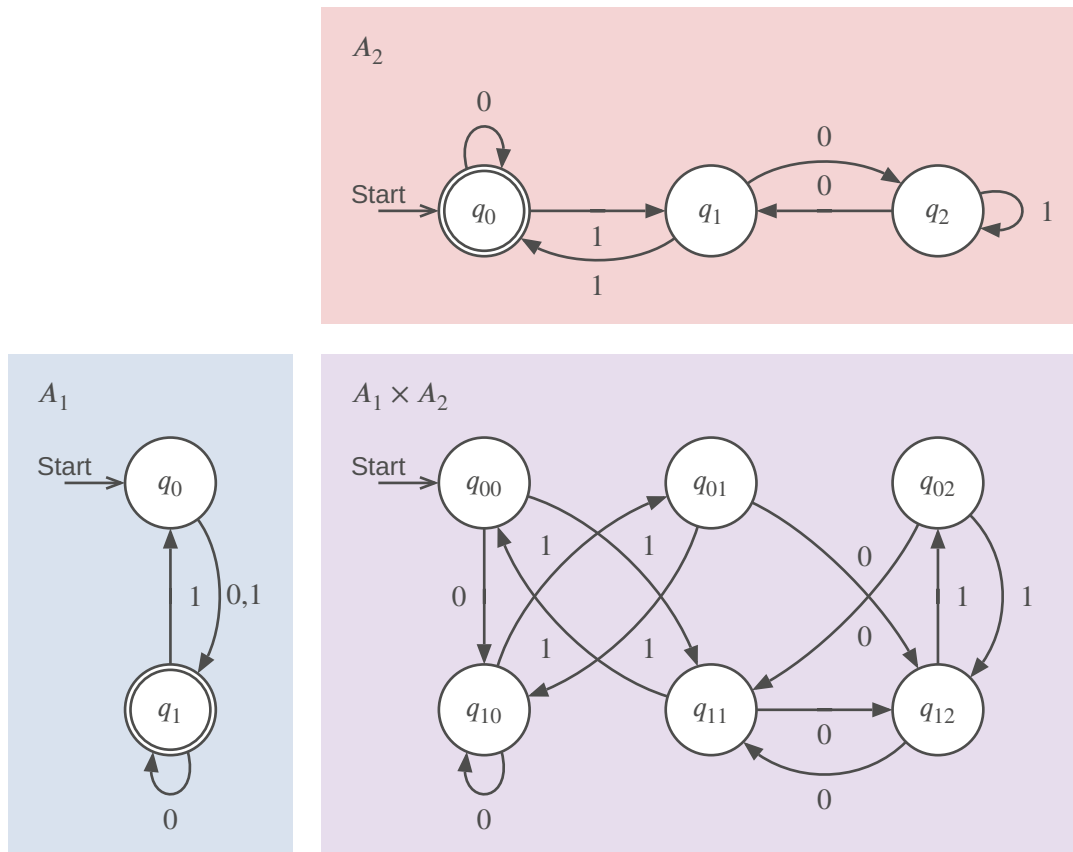


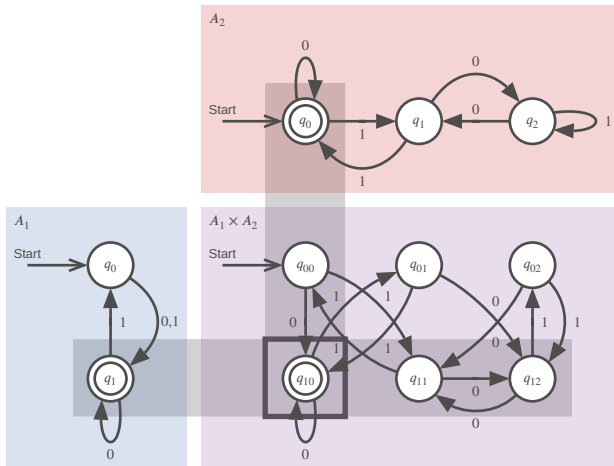
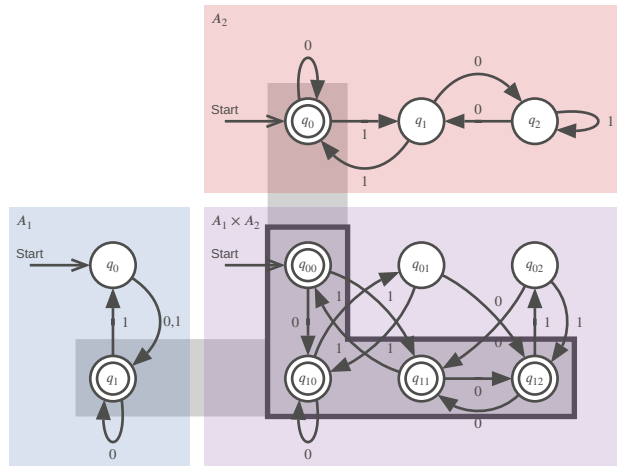
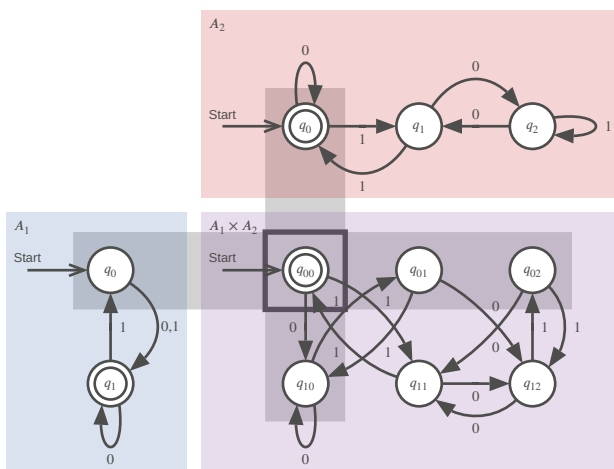
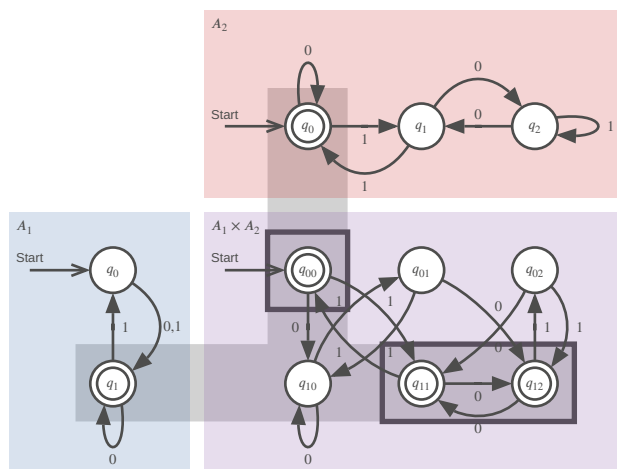
## 4.8. MENGENOPERATIONEN

Lassen reguläre Sprachen regulär sein.

Produktautomat:  $L_i = L(A_i)$

$$L = \{w \in \Sigma^* \mid |w|_1 \text{ ungerade}\} \cap \{w \in \Sigma^* \mid w \text{ ist eine durch drei teilbare Binärzahl}\}$$



Schnittmenge  $L(A_1) \cap L(A_2)$ Vereinigungsmenge  $L(A_1) \cup L(A_2)$ Differenzmenge  $L(A_1) \setminus L(A_2)$ Symmetrische Differenz  $L(A_1) \triangle L(A_2)$ 

Operationen verändern nur Endzustände:

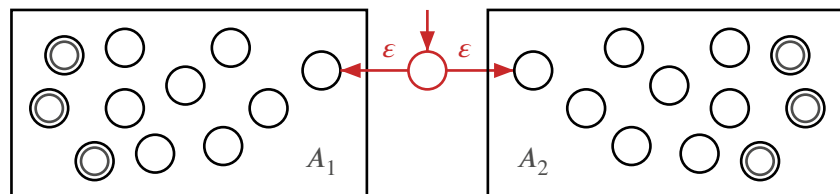
| Begriff                       | Bedeutung                                |
|-------------------------------|------------------------------------------|
| Schnittmenge $L_1 \cap L_2$   | $F = F_1 \times F_2$                     |
| Vereinigung $L_1 \cup L_2$    | $F = F_1 \times Q_2 \cup Q_1 \times F_2$ |
| Differenz $L_1 \setminus L_2$ | $F = F_1 \times (Q_2 \setminus F_2)$     |

## 4.9. REGULÄRE OPERATIONEN

WICHTIG:  $\epsilon$ -Übergänge immer hinzufügen

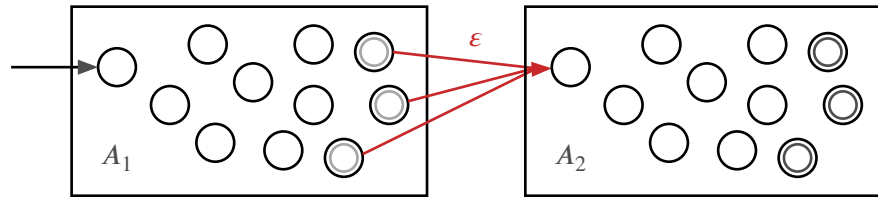
### 4.9.1. Alternative

$$\begin{aligned}
 L &= L_1 \cup L_2 \\
 &= L(A_1) \cup L(A_2) \\
 &= L(A_1) \mid L(A_2)
 \end{aligned}$$



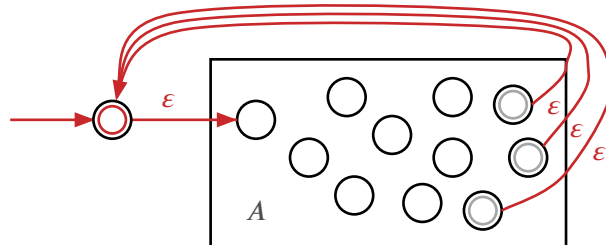
### 4.9.2. Verkettung

$$\begin{aligned}
 L &= L_1 L_2 \\
 &= L(A_1) L(A_2) \\
 &= \{w_1 w_2 \mid w_i \in L_i\}
 \end{aligned}$$



### 4.9.3. \*-Operation

$$\begin{aligned}
 L^* &= \{\epsilon\} \cup L \cup L^2 \cup \dots \\
 &= \bigcup_{k=0}^{\infty} L^k
 \end{aligned}$$



⇒ die Klasse der regulären Sprachen ist abgeschlossen unter regulären Operationen

## 4.10. REGULÄRE AUSDRÜCKE

Formeln, die Zeichenketten beschreiben.

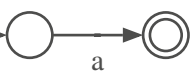
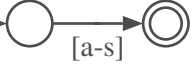
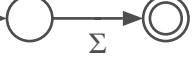
- 1) Buchstaben stehen für sich selbst, mit Ausnahme der Metazeichen  $()[]*?|.\backslash$  (escape character)
- 2) Verkettung: Zeichen und Formeln hintereinanderschreiben
- 3)  $.$  = ein beliebiges Zeichen,  $\Sigma$
- 4)  $|$ : Alternative,  $a|A = a$  oder  $A$
- 5)  $*$ : Wiederholung, beliebige viele
- 6)  $()$ : Gruppierung
- 7)  $[]$  Zeichenklassen:  $[abc] = a|b|c$ ,  $[\wedge abc] = \text{nicht } [abc]$

Erweiterungen / Dialekte

- 1)  $\{n, m\}$ : zwischen  $n$  und  $m$  Wiederholungen
- 2)  $+$ : mindestens eines,  $\{1, \infty\}$
- 3)  $?$ : optional,  $\{0, 1\}$
- 4) Symbole für Zeichenklassen:  $\backslash d$  Ziffern,  $\backslash s$  whitespace,  $[: space :]$ ,  $[: lower :]$

### 4.10.1. Verallgemeinerter NEA (VNEA)

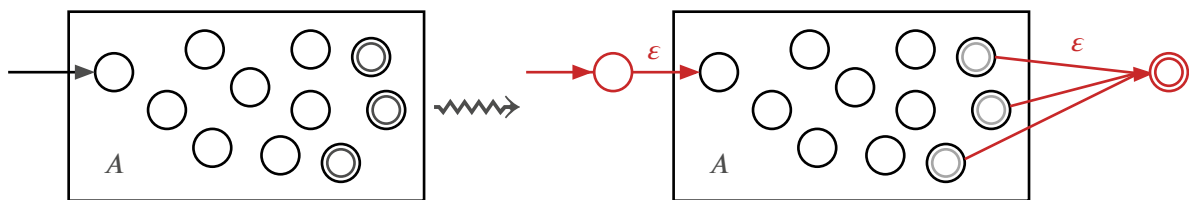
| Begriff               | Bedeutung                                                                              |
|-----------------------|----------------------------------------------------------------------------------------|
| Regulärer Ausdruck    | Zeichenkette $r$ zur Beschreibung einer regulären Sprache $L = L(r)$                   |
| Reguläre Operationen  | $L(r_1) \cup L(r_2) = L(r_1   r_2)$ $L(r_1) L(r_2) = L(r_1 r_2)$ $L(r_1)^* = L(r_1^*)$ |
| Verallgemeinerter NEA | $\text{NEA}_\epsilon$ , dessen Übergänge mit regulären Ausdrücken beschriftet sind     |

| Begriff                      | Bedeutung                                        |     |                                                                                                        |
|------------------------------|--------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------|
| Primitive reguläre Ausdrücke | Reguläre Ausdrücke für Wörter mit Länge $\leq 1$ |     |                                                                                                        |
|                              | $L = L(r)$                                       | $r$ | NEA                                                                                                    |
| $\emptyset$                  | $\emptyset$                                      |     | Start $\rightarrow$   |
| $\{\varepsilon\}$            | $\{\varepsilon\}$                                |     | Start $\rightarrow$   |
| $\{a\}$                      | $a$                                              |     | Start $\rightarrow$  |
| $\{o, s, t\}$                | $[ost]$                                          |     | Start $\rightarrow$  |
| $\{a, b, \dots, s\}$         | $[a-s]$                                          |     | Start $\rightarrow$  |
| $\Sigma$                     | $.$                                              |     | Start $\rightarrow$  |

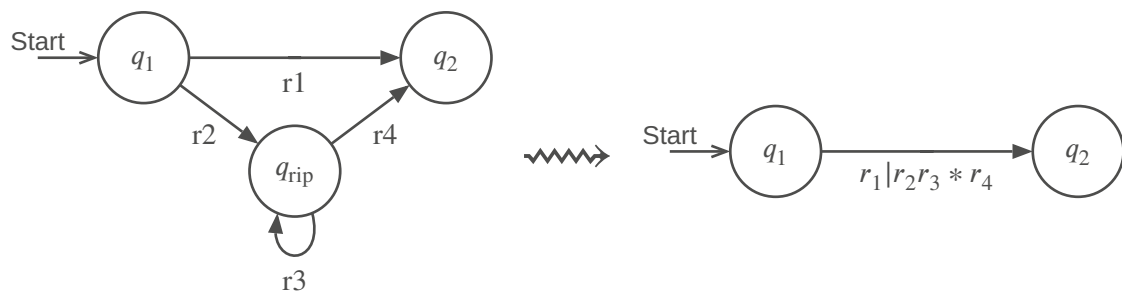
$\Rightarrow$  zu jedem regulären Ausdruck gibt es einen DEA

#### 4.10.1.1. VNEA $A$ umwandeln in Regex $r$

**Keine Übergänge nach  $q_0$  und nur ein Akzeptierzustand**



#### Reduktion



#### Regulärer Ausdruck

Nach Entfernen aller Zwischenzustände  $q_{rip} \in Q$  bleibt ein regulärer Ausdruck  $r$  von  $A$



$\Rightarrow$  jede reguläre Sprache lässt sich mit einem regulären Ausdruck beschreiben  $\{L(A) \mid A \text{ ein DEA}\} = \{L(r) \mid r \text{ ein regulärer Ausdruck}\}$

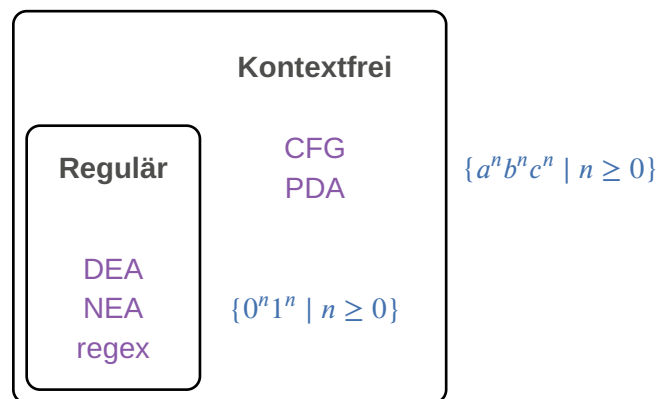
Interessantes projekt (DEA Lexer DSL): [Ragel](#)

### 4.10.2. Teststrategie

| Messung                                                                                                                                                                                                                                                                                                      | Folgerung                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <p>Für <math>n = 1, 2, 3, \dots</math></p> <p>– Regulären Ausdruck erzeugen<br/> <math>r = \underbrace{a^? a^? a^? a^? \dots a^? a^? a^? a^?}_{n \cdot a^?} \dots \underbrace{a^? a^? a^? a^? a^?}_{n \cdot a}</math></p> <p>– Laufzeit für Akzeptieren von <math>a^n</math> durch <math>r</math> messen</p> | <p>– Laufzeit <math>O(2^n)</math>: NEA-Implementation</p> <p>– Laufzeit <math>O(n)</math>: DEA-Implementation</p> |

## 4.11. KONTEXTFREIE SPRACHEN

| Begriff | Bedeutung             |
|---------|-----------------------|
| CFL     | Context Free Language |
| CFG     | Context Free Grammar  |
| PDA     | Pushdown Automata     |



### 4.11.1. Kontextfreie Grammatik und Sprache

**Kontextfreie Grammatik:**  $G = (V, \Sigma, R, S)$

- $V$ : Variablen
- $\Sigma$ : Terminalsymbole (Alphabet)
- $R$ : Regeln der Form  $A \rightarrow x_1 x_2 \dots x_n$  mit  $A \in V, x_i \in V \cup \Sigma$
- $S$ : Startvariable

#### Ableitung und erzeugte Sprache

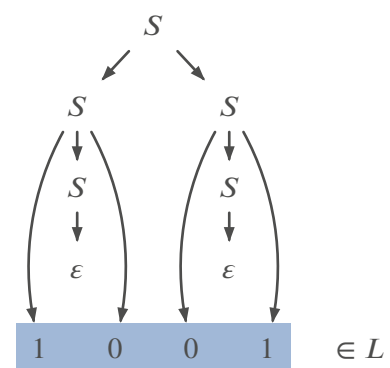
- Regel  $A \rightarrow w$  erzeugt aus  $uAv$  das Wort  $uwv$ :  $uAv \Rightarrow^* uwv$
- $v$  aus  $u$  ableiten:  $u \Rightarrow^* u_1 \Rightarrow^* u_2 \Rightarrow^* \dots \Rightarrow^* u_n \Rightarrow^* v$  oder  $u \Rightarrow^* v$
- $L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$  von  $G$  erzeugte kontextfreie Sprache

#### Parse Tree

$$L = \{w \in \{0, 1\}^* \mid |w|_0 = |w|_1\}$$

mit der Grammatik

$$S \rightarrow 0S1 \mid 1S0 \mid SS \mid \epsilon$$



**Beispiel 8:**  $L = \{0^n 1^n \mid n \geq 0\}$

Grammatik  $L = \{0^n 1^n \mid n \geq 0\}$

Wörter, die erzeugt werden müssen

1) Variablen:  $V = \{S\}$

2) Terminalsymbole:  $\Sigma = \{0, 1\}$

3) Regeln:  $R = \{S \rightarrow \epsilon \mid 0S1\}$

4) Startvariable:  $S$

$\epsilon$

$01 = 0\epsilon 1$

$0011 = 0011$

#### 4.11.2. Mehrere Grammatiken

**TODO:**

Eindeutigkeit

#### 4.11.3. Reguläre Operationen

##### Grammatik für reguläre Operationen

$L_1$  und  $L_2$  kontextfreie Sprachen mit Grammatiken  $G_i = (V_i, \Sigma, R_i, S_i)$ .

Grammatik für reguläre Operationen:

- Neue Startvariable  $S_0$
- $V = V_1 \cup V_2 \cup \{S_0\}$
- geeignet erweiterte Regeln  $R \Rightarrow G = (V, \Sigma, R, S_0)$

##### Satz

Die Klasse der kontextfreien Sprachen ist abgeschlossen unter regulären Operationen.

**Reguläre Sprachen sind kontextfrei.**

##### Alternative

Regeln für  $L_1 \cup L_2$ :

$$R = R_1 \cup R_2 \cup \{S_0 \rightarrow S_1, S_0 \rightarrow S_2\}$$

##### Verkettung

Regeln für  $L_1 L_2$ :

$$R = R_1 \cup R_2 \cup \{S_0 \rightarrow S_1 S_2\}$$

##### \*-Operation

Regeln für  $L_1^*$ :

$$R = R_1 \cup \{S_0 \rightarrow S_0 S_1, S_0 \rightarrow \epsilon\}$$

#### 4.11.4. Kontextfrei

##### Regeln ohne Kontext

$$S \rightarrow A \mid C$$

$$A \rightarrow a$$

$$A \rightarrow aAb$$

$$C \rightarrow bA$$

##### Regeln mit Kontext

$$S \rightarrow C \mid aC \mid bC$$

$$aC \rightarrow A$$

$$bC \rightarrow B$$

Es spielt keine Rolle, in welchem Kontext das Zeichen  $A$  vorkommt, die Regel kann immer angewendet werden

$$S \rightarrow aAb \rightarrow aab$$

Je nach **Kontext** kann eine Regel nicht unbedingt angewendet werden

$$S \rightarrow aC \rightarrow A$$

$$S \rightarrow bC \rightarrow B$$

$$S \rightarrow C \rightarrow ?$$

**Python ist nicht kontextfrei** lmao

#### 4.11.5. Ableitung

##### Probleme

1) Unit-rules

$$\left. \begin{array}{l} A \rightarrow B \\ B \rightarrow A \\ A \rightarrow a \end{array} \right\} \Rightarrow A \rightarrow B \rightarrow \underbrace{A \rightarrow B \rightarrow \dots \rightarrow A}_{\text{beliebig lange}} \rightarrow a$$

##### Lösung

- 1) Keine Unit-rules  $A \rightarrow B$
- 2) Keine Regeln  $A \rightarrow \epsilon$  ausser wenn nötig  $S \rightarrow \epsilon$
- 3) Keine Regeln mit mehr als 2 Variablen auf der rechten Seite

## 2) «Aufblasen» und «Luft rauslassen»

Genauer: rechte Seite enthält genau zwei Variablen oder genau ein Terminalsymbol

$$\left. \begin{array}{l} A \rightarrow ABCD \mid a \\ B \rightarrow \varepsilon \\ C \rightarrow \varepsilon \\ D \rightarrow \varepsilon \end{array} \right\} \Rightarrow A \rightarrow ABCD \rightarrow ABC \rightarrow AB \rightarrow A \rightarrow a$$

## 4.11.6. Chomsky-Normalform (CNF)

Eine CFG ist in **Chomsky-Normalform**, wenn  $S$  auf der rechten Seite nicht vorkommt und jede Regel von der Form  $A \rightarrow BC$  oder  $A \rightarrow a$  ist, zusätzlich ist die Regel  $S \rightarrow \varepsilon$  erlaubt.

## 4.11.6.1. Umwandlung

- 1) Neue Startvariable  $S_0 \rightarrow S$  (wenn nötig)
- 2)  $\varepsilon$ -Regeln:  $\left. \begin{array}{l} A \rightarrow \varepsilon \\ B \rightarrow AC \end{array} \right\} \Rightarrow A$  kann weggelassen werden  $\Rightarrow \left\{ \begin{array}{l} B \rightarrow AC \\ \quad \rightarrow AC \end{array} \right.$
- 3) Unit-Rules:  $\left. \begin{array}{l} A \rightarrow B \\ B \rightarrow CD \end{array} \right\} \Rightarrow$  aus  $A$  kann man wie aus  $B$  auch  $CD$  machen  $\Rightarrow \left\{ \begin{array}{l} A \rightarrow CD \\ B \rightarrow CD \end{array} \right.$
- 4) Verkettungen:  $A \rightarrow u_1 u_2 \dots u_n$  ersetzen durch  $A \rightarrow u_1 A_1, A_1 \rightarrow u_2 A_2, \dots, A_{n-2} \rightarrow u_{n-1} u_n$  und falls  $u_i$  ein Terminalsymbol ist:  $A_{i-1} \rightarrow U_i A_i, U_i \rightarrow u_i$ .

## 4.11.6.2. Folgerungen

- 1) Ableitung eines Wortes  $w \in L(G)$  ist immer in  $2|w| - 1$  Regelanwendungen möglich. Beweis:
  - $|w| - 1$  Regeln der Form  $A \rightarrow BC$  um aus  $S$  ein Wort aus  $|w|$  Variablen zu erzeugen
  - $|w|$  Regeln der Form  $A \rightarrow a$  um das Wort  $w$  zu erzeugen
  - $\Rightarrow$  insgesamt  $2|w| - 1$  Regelanwendungen
- 2) Deterministischer Parse-Algorithmus mit Laufzeit  $O(|w|^3)$

$$S \rightarrow ASA \mid aB$$

**Beispiel 9:**  $A \rightarrow B \mid S$

$$B \rightarrow b \mid \varepsilon$$

## 1) Startvariable

$$\begin{array}{l} S_0 \rightarrow S \\ S \rightarrow ASA \mid aB \\ A \rightarrow B \mid S \\ B \rightarrow b \mid \varepsilon \end{array}$$

2)  $\varepsilon$ -Regel

$$\begin{array}{ll} \begin{array}{l} S_0 \rightarrow S \\ S \rightarrow ASA \mid aB \mid a \\ A \rightarrow B \mid \varepsilon \mid S \\ B \rightarrow b \end{array} & \rightsquigarrow \begin{array}{l} S_0 \rightarrow S \\ S \rightarrow ASA \mid SA \mid AS \mid aB \mid a \\ A \rightarrow B \mid S \\ B \rightarrow b \end{array} \end{array}$$

## 3) Unit-Regel

$$\begin{array}{ll} \begin{array}{l} S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a \\ S \rightarrow ASA \mid SA \mid AS \mid aB \mid a \\ A \rightarrow B \mid ASA \mid SA \mid AS \mid aB \mid a \\ B \rightarrow b \end{array} & \rightsquigarrow \begin{array}{l} S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a \\ S \rightarrow ASA \mid SA \mid AS \mid aB \mid a \\ A \rightarrow b \mid ASA \mid SA \mid AS \mid aB \mid a \\ B \rightarrow b \end{array} \end{array}$$

## 4) Komplexe Regeln

$$S_0 \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$$



$$\begin{aligned}
 S &\rightarrow AA_1 \mid SA \mid AS \mid UB \mid a \\
 A &\rightarrow b \mid AA_1 \mid SA \mid AS \mid UB \mid a \\
 A_1 &\rightarrow SA \\
 U &\rightarrow a \\
 B &\rightarrow b
 \end{aligned}$$

#### 4.11.7. Cocke-Younger-Kasami Algorithmus (CYK)

##### Gegeben

- 1) Grammatik  $G = (V, \Sigma, R, S)$
- 2) Variable  $A \in V$
- 3) Wort  $w \in \Sigma^*$

##### Frage

Ist  $w$  ableitbar? in Zeichen  $A \Rightarrow^* w$

- Spezialfall  $w = \varepsilon$ :  $A \Rightarrow^* \varepsilon \Leftrightarrow A \rightarrow \varepsilon \in R$  (Epsilonregel,  $A = S$ )
- Spezialfall  $|w| = 1$ :  $A \Rightarrow^* w \Leftrightarrow A \rightarrow w \in R$  (Terminalsymbolregel)
- Fall  $|w| > 1$ :

$$A \Rightarrow^* w \Rightarrow \exists \left\{ \begin{array}{l} A \rightarrow BC \in R \\ w = w_1 w_2 \\ w_i \in \Sigma^* \end{array} \right. \text{ mit } \left\{ \begin{array}{l} B \Rightarrow^* w_1 \\ C \Rightarrow^* w_2 \end{array} \right.$$

##### 4.11.7.1. Ableitungsdreieck

##### Ideen

- Parse Tree aus  $A \rightarrow BC$  und  $T \rightarrow t$
- Variablen in Tabelle füllen

##### Prinzip

- Einem Teilwort entspricht ein Feld der Tabelle (rot hinterlegt)
- Das Feld wird mit den Variablen gefüllt, aus denen das Teilwort abgeleitet werden kann.

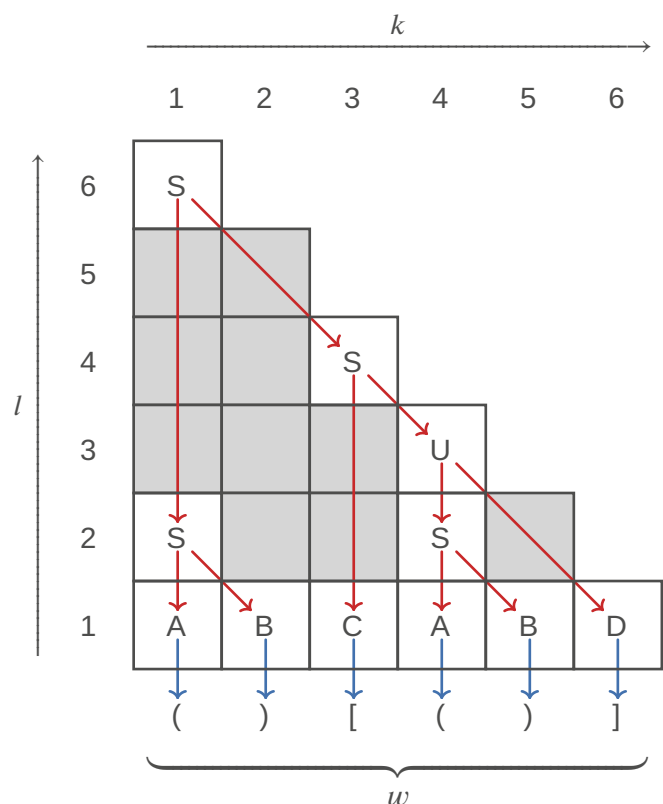
##### Beispiel

Parse des Wortes  $() [ ( ) ]$

$$\begin{array}{ll}
 S \rightarrow AB \mid CD \mid CU \mid SS & B \rightarrow ) \\
 U \rightarrow SD & C \rightarrow [ \\
 A \rightarrow ( & D \rightarrow ]
 \end{array}$$

##### Definitionen

Die Felder  $(k, l_1)$  und  $(k + l_1, l - l_1)$  heissen **korrespondierende Felder** im Ableitungsdreieck des Feldes  $(k, l)$ .



## 5. STACK

Unendlich grosser Speicher

- Immer nur das oberste Element sichtbar
- Beliebig tief

## 5.1. STACKAUTOMAT / PUSHDOWN AUTOMATON (PDA)

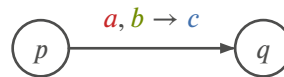
Kann Sprachen akzeptieren, die nicht regulär sind, ist aber meistens nicht deterministisch.

### Definition

Stackautomat  $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$

- 1)  $Q$ : Zustände
- 2)  $\Sigma$ : Eingabe-Alphabet
- 3)  $\Gamma$ : Stack-Alphabet
- 4)  $\delta: Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow P(Q \times \Gamma_\epsilon)$
- 5)  $q_0 \in Q$ : Startzustand
- 6)  $F \subset Q$ : Akzeptierzustände

### Übergänge



$a$  vom Input

$b$  vom Stack entfernen (Bedingung)

$c$  auf den Stack

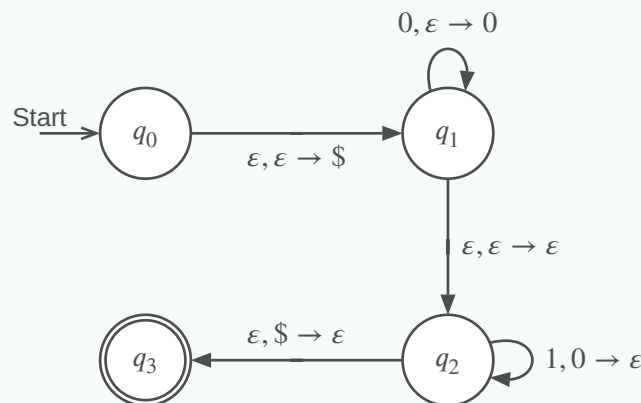
### Beachte

- immer nicht-deterministisch
- $\Gamma \neq \Sigma$  möglich

### TODO:

finish

### Beispiel 10: Stackautomat für die Sprache $\{0^n 1^n \mid n \geq 0\}$



### TODO:

Book 167-168, shift/reduce

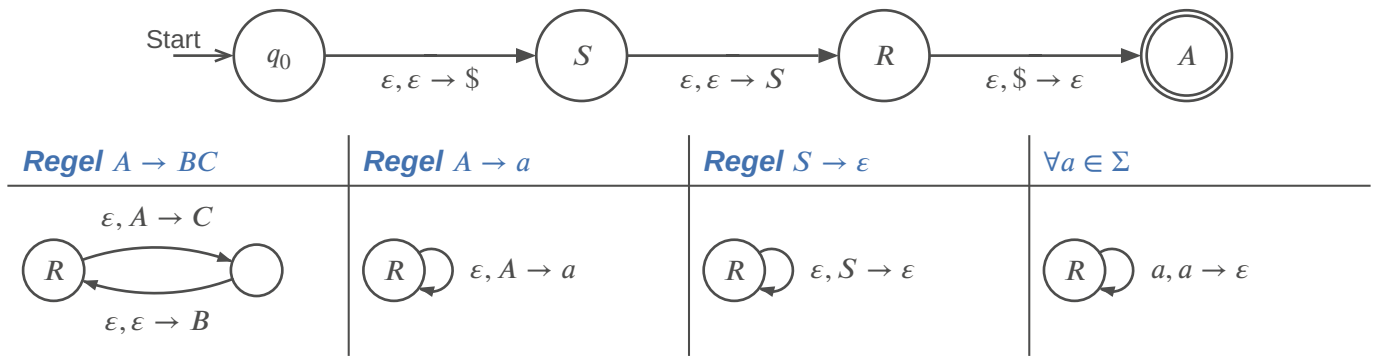
### TODO:

diagrams? (slides 8/12)

### 5.1.1. Ableitung aus CNF

Ist  $L$  eine kontextfreie Sprache, dann gibt es einen Stackautomaten  $P$ , der  $L$  akzeptiert,  $L = L(P)$ .

### Grundgerüst



### 5.1.2. PDA $\rightarrow$ CFG

#### Variablen

Wörter beschreiben Pfade durch den Automaten: Variable  $A_{pq}$  = Wörter, die von  $p$  nach  $q$  führen mit leerem Stack

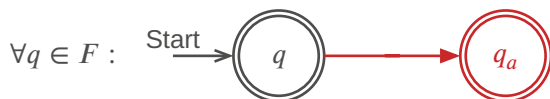
#### Regeln

Regeln beschreiben, wie sich Wege zerlegen lassen:  $A_{pq} \rightarrow A_{pr}A_{rq}$

«Wege von  $p$  nach  $q$  können verstanden werden als Wege von  $p$  nach  $r$  und von dort nach  $q$ »

#### 5.1.2.1. Stackautomat standardisieren

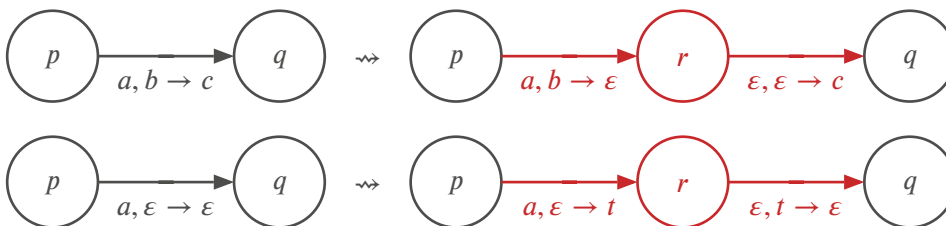
1) Nur ein Akzeptierzustand: neuer Akzeptierzustand  $q_a$  und Übergänge



2) Stack leeren



3) Jeder Übergang legt entweder ein Zeichen auf den Stack oder entfernt eines



#### 5.1.2.1.1. Regeln

**TODO:**

slides

**TODO:**

Book 182,183

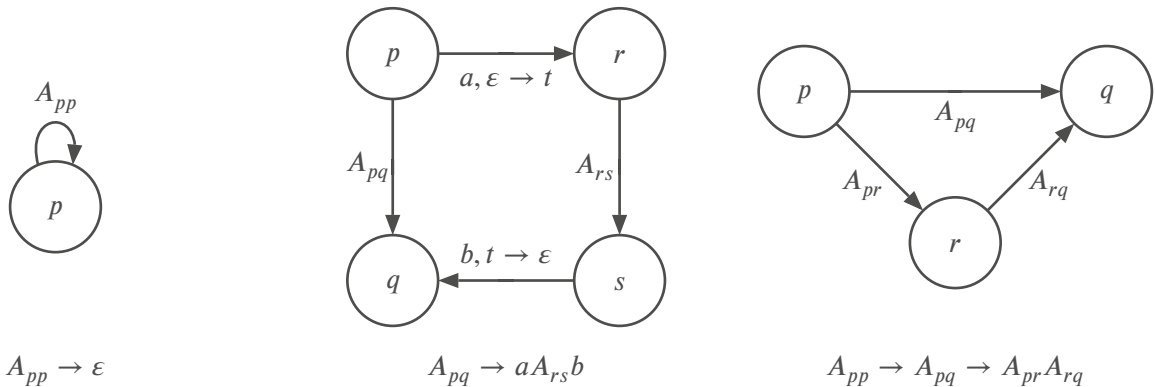
#### Beispiel 11

**TODO:**  
visualization

#### 5.1.2.1.2. Grammatik ablesen

Ausgangspunkt: standardisierter PDA mit Startzustand  $q_0$  und  $F = \{q_a\}$ .

- 1) Startvariable:  $A_{q_0, q_a}$
- 2) Regeln:



#### Beispiel 12

**TODO:**

#### 5.1.2.2. Pumping Lemma

##### Pumping Lemma für CFL

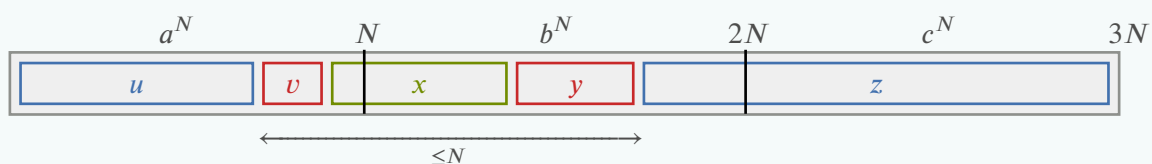
Ist  $L$  eine CFL, dann gibt es eine Zahl  $N$ , die Pumping Length, derart, dass jedes Wort  $w \in L$  mit  $|w| \geq N$  zerlegt werden kann in fünf Teile  $w = uvxyz$  derart, dass

- 1)  $|vy| > 0$
- 2)  $|vxy| \leq N$
- 3)  $uv^kxy^kz \in L \forall k \in \mathbb{N}$

Mit dem Pumping Lemma kann man beweisen, dass eine Sprache **nicht** kontextfrei ist.

#### Beispiel 13: $\{a^n b^n c^n \mid n \geq 0\}$

- 1) Annahme:  $L$  kontextfrei
- 2) Pumping length  $N$
- 3) Wort:  $w = a^N b^N c^N$
- 4) Zerlegungen:



- 5) Beim Pumpen nimmt die Anzahl der  $a$  und  $b$  zu, nicht aber die Anzahl der  $c$   
 $\Rightarrow uv^kxy^kz \notin L \forall k \neq 1$
- 6) Widerspruch:  $L$  nicht kontextfrei

#### 5.1.2.2.1. Herleitung

##### Grammatik $G$ in CNF

$$w \in L(G) \Rightarrow S \Rightarrow^* w$$

##### Wiederverwendete Variable

$|w| \geq N$  gross genug  $\Rightarrow$  Variablen werden im Parse Tree wiederverwendet Die «unterste» wiederverwendete Variable  $A$  erzeugt zwei Wörter:

$$A \Rightarrow^* vxy$$

$$A \Rightarrow^* x$$

##### Pumpen

$A \Rightarrow^* vxy$  anstelle des «untersten»  $A \Rightarrow^* x$  verwenden

**TODO:**  
 diagram

## 6. PARSING

### 6.1. BACKUS-NAUR-FORM (BNF)

Spezifikation der Regeln in maschinen-lesbarer Form:

- Variablen: **<variablen-name>**
- Einzelne Zeichen:  $A$
- Zeichenketten: 'BEISPIEL'
- Regeln: **<variablen-name> ::= Ausdruck**
- Ausdrücke sind Folgen von Variablen, einzelnen Zeichen oder Zeichenketten, getrennt durch  $|$

#### Beispiel 14 : BNF für Expression-Term-Factor Grammatik

##### Ausgangsgrammatik

```

expression → expression + term | term
term → term * factor | factor
factor → (expression) | N
N → NZ | Z
Z → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

```

##### Backus-Naur-Form

```

<expression> ::= <expression> + <term> | <term>
<term>       ::= <term> * <factor> | <factor>
<factor>     ::= ( <expression> ) | <number>
<number>     ::= <number> <digit> | <digit>
<digit>      ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

```

## 6.2. EXTENDED BACKUS-NAUR-FORM (EBNF)

### Definition

- Literale in Anführungszeichen oder Apostroph
- = statt :: =
- Variablen ohne < und >, dürfen Leerzeichen enthalten
- Komma für Verkettung ,
- Regel-Endzeichen ;
- Kommentare (\* ... \*)
- Optionale Wiederholung { ... }
- Option [ ... ]
- Gruppierung ( ... )
- Ausnahme: -

### Achtung!

- Keine Operatorpriorisierung
- Zweideutig!
- Keine eindeutige Evaluation!

### Beispiel 15: EBNF für Expression-Term-Factor Grammatik

```

expression = expression, "+", term | term ;
term       = term, "*", factor, | factor ;
factor     = "(", expression, ")" | number ;
number     = digit, { digit } ;
digit      = "0" | "1" | "2" | "3" | "4" |
            "5" | "6" | "7" | "8" | "9" ;

```

## 6.3. RECHTSREKURSION

Expression-Term-Factor-Grammatik verwendet Links-Rekursion  $\Rightarrow$  korrekte Auswertung von links nach rechts. Parsetree wertet aber Ausdrücke von rechts nach links aus! Alternative Expression-Term-Factor definition mit Rechtsrekursion statt Linksrekursion:

```

expression  $\rightarrow$  term expression'
expression'  $\rightarrow$  + term expression' |  $\epsilon$ 
term  $\rightarrow$  factor term'
term'  $\rightarrow$  * factor term' |  $\epsilon$ 
factor  $\rightarrow$  (expression) | N
N  $\rightarrow$  NZ | Z
Z  $\rightarrow$  0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

```

### TODO:

grid  $\rightarrow$  table

## 7. UNENDLICH

| Begriff                 | Bedeutung                                                                                                                                                                         |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Abzählbar unendlich     | Eine Menge $A$ heisst <b>abzählbar unendlich</b> , wenn es eine Bijektion $\mathbb{N} \rightarrow A$ gibt, also $A \simeq \mathbb{N}$ . Bsp: $\mathbb{R}, \mathbb{Z}, \mathbb{Q}$ |
| Überabzählbar unendlich | Eine Menge $A$ heisst <b>überabzählbar unendlich</b> , wenn sie nicht abzählbar unendlich ist. Bsp: $\mathbb{R}, \mathbb{C}, P(\mathbb{N})$                                       |
| Gleich mächtig          | Mengen $A$ und $B$ heissen <b>gleich mächtig</b> , $A \simeq B$ , wenn es eine Bijektion $A \rightarrow B$ gibt                                                                   |
| Unendlich               | Eine Menge $A$ heisst <b>unendlich</b> , wenn sie gleich mächtig wie eine Teilmenge ist                                                                                           |

$A, B, A_k$  abzählbar unendlich,  $k \in \mathbb{N}$ :

- 1)  $A \cup B, \bigcup_k A_k$  abzählbar unendlich
- 2)  $A \times B$  abzählbar unendlich
- 3) Abzählbar unendlich:  $\mathbb{N}, \mathbb{Z}, \mathbb{Q}$
- 4)  $P(A)$  überabzählbar unendlich
- 5)  $\mathbb{R}$  überabzählbar unendlich

- Abzählbar unendlich:  $\Sigma^*$ , Menge aller DEAs/NEAs/PDAs/CFGs
- Überabzählbar unendlich: Menge aller Sprachen  $P(\Sigma^*)$

### TODO:

Credits: Prof. Dr. Andreas Müller